

131. Des ennemis qui vivent et qui meurent

Les méthodes, dans un vrai tour de boucle.

1. Ce qu'il faut comprendre

`hasard()` lit le compteur libre du matériel : un nombre imprévisible, sans table ni graine. `semer(n)` permet de choisir son point de départ quand on veut au contraire une suite reproductible.

`cache(n)` ôte le lutin de l'écran. Sans lui, un ennemi mort resterait affiché, immobile — ce qui se remarque tout de suite, heureusement.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <cache>` → [0.107.3. La fonction cache\(\) — ôter un lutin](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **710 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	56 o	← la console, à l'allumage — la mise en route de la console
Monstre::vivant()	21 o	← Monstre::avancer(), main() — ta fonction
Monstre::avancer()	35 o	← main() — ta fonction
Monstre::tuer()	13 o	← main() — ta fonction
main()	378 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
Reste	10 o	← main() — le reste d'une division
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
Hasard	25 o	← hasard() — tire un nombre au hasard
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	16 o	← CopierTuiles — 0 lettre sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite()	x 1	88 o sur place, dans main()
cache()	x 1	13 o sur place, dans main()
bouton()	x 1	12 o sur place, dans main() + LireManette (37 o)
image()	x 1	6 o sur place, dans main() + AttendreImage (28 o)

```
hasard()          x 1      3 o sur place, dans main() + Hasard (25 o)
Tes fonctions appelées :
Monstre::vivant() x 1      15 o sur place, dans Monstre::avancer()
```

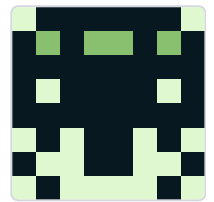
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois monstres avancent. Chaque appui sur A en tue un au hasard :
// il disparaît de l'écran.
//
// Chaque monstre est une struct avec des méthodes (vivant, avancer, tuer).
// Un monstre mort ne s'affiche plus : cacher(numéro) retire un lutin de l'écran.

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <bouton>     // lit un bouton de la manette
#include <hasard>     // tire un nombre au hasard
#include <sprite>     // place un lutin de 8 × 8 au pixel près
#include <cacher>     // cache un lutin
#include <reste>      // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

2. UN DESSIN — ENNEMI

```
Tuile ENNEMI = {
    ".#####.", "#-#--#-", "#####", "#.####.#",
    "#####", ".#.#.#.", "#..##..#", ".#....#.",
};
```



à l'écran

3. UN ENREGISTREMENT — MONSTRE

```
struct Monstre {
    uint8_t x, y, vie;                // sa place (en pixels) et sa vie

    uint8_t vivant() { return vie > 0; } // 1 s'il vit
    void avancer() { if (vivant()) x++; } // un pas à droite, s'il vit
    void tuer() { vie = 0; }           // plus de vie
};
```

4. UN TABLEAU — TROUPE

```
Monstre troupe[3];                    // 3 monstres.
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                     // Le jeu commence ici.
    for (uint8_t i = 0; i < 3; i++) {           // Chaque monstre i (0, 1, 2) :
        troupe[i].x = 10 + i * 30;             // x = 10, 40, 70
        troupe[i].y = 40 + i * 24;             // y = 40, 64, 88
        troupe[i].vie = 1;                     // vivant.
    }

    uint8_t avant = 0;                          // A au tour d'avant.

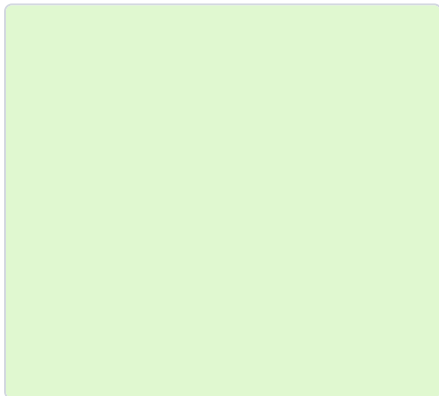
    while (true) {                              // Répète pour toujours :
        uint8_t appui = bouton(A);
        if (appui && !avant) troupe[hasard() % 3].tuer(); // Début d'appui : on tue le
                                                    // monstre n° 0, 1 ou 2, au hasard.

        avant = appui;

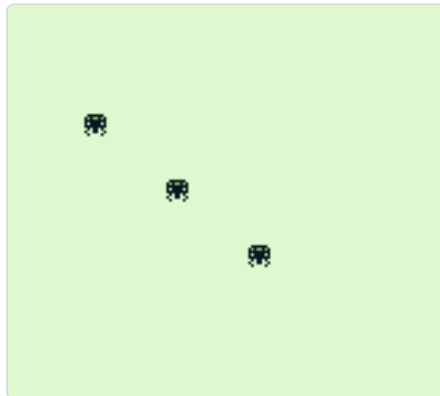
        for (uint8_t i = 0; i < 3; i++) {       // Pour chaque monstre :
            troupe[i].avancer();                 // il avance (s'il vit),
            if (troupe[i].vivant()) sprite(i, troupe[i].x, troupe[i].y, ENNEMI); // vivant : on le montre
            else cacher(i);                      // mort : on cache son lutin.
        }
        image();                                // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

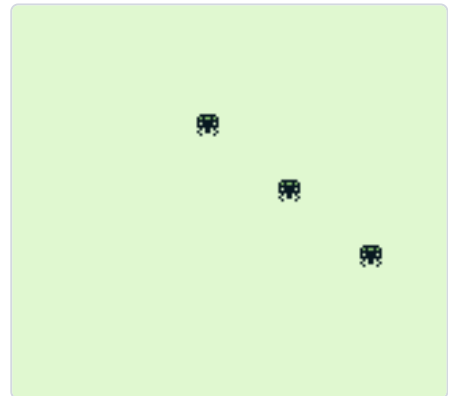
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



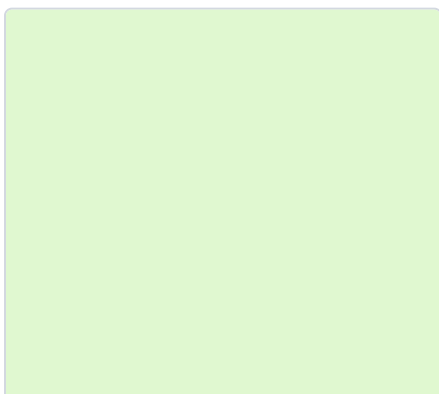
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé

5. Ce qu'on doit voir

Trois monstres qui avancent ; A en fait disparaître un au hasard.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les trois sont posés
- ✓ ils avancent
- ✓ A finit par tous les tuer (0 encore à l'écran)
- ✓ et cacher() les a bien ôtés de l'écran

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

ENNEMI

+ tuile 8 × 8

+ perso 16 × 16

+ perso 32 × 32

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

ENNEMI

tuile n° 44 — poser(x, y, ENNEMI)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

×2

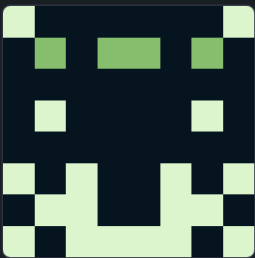
÷2

Agrandir...

Vider le dessin

Taille voulue :

✓ Créer la copie



taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

ENNEMI ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

ENNEMI peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

Ensuite : **132. Les collisions** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++